

Data lake performance

Partitioning & bucketing

Lesson objectives

Data lake performance: Partitioning
& bucketing

1. Explain the role that partitioning plays in data lakes.
2. Explain the role that bucketing plays in data lakes.
3. Explain how sorting and bloom filters provide additional sorting strategies.

Partitioning

Data lakes use partitioning as an alternative to indexing

- A partition is just a way to divide data into subdirectories that are designed around logical groupings.
- **Example:** Suppose you have 5 years of historical data.
 - Rather than dump all 5 years into a single folder, you might create a folder for each year, and then put each year's data into the appropriate folder.
 - When a query includes the partition key in the predicate or join, the query engine can avoid reading partitions that don't match.
- **Number of partitions not fixed:** Partitions are added as new data is added (IE: New year = new partition).

```
.../cat/sch/hist_tbl
|-- year=2020
|   |-- file1
|   |-- file2
|-- year=2021
|   |-- file3
|   |-- file4
|   |-- file5
|-- year=2022
|   |-- file6
|   |-- file7
```

Define partition columns at table creation

```
CREATE TABLE mycat.myschema.logs_daily_part_orc (  logs_daily_part_orc
```

```
    event_time      TIMESTAMP,  
    ip_address      VARCHAR(15),  
    app_name        VARCHAR(25),  
    process_id      SMALLINT,  
    log_type        VARCHAR(15),  
    log_level       VARCHAR(15),  
    message_id      VARCHAR(15),  
    message_details VARCHAR(555),  
    log_date        CHAR(10)  
  ) WITH (  
    format = 'ORC',  
    partitioned_by = ARRAY['log_date']  
  );
```

	event_time	timestamp(3)
	ip_address	varchar(15)
	app_name	varchar(25)
	process_id	smallint
	log_type	varchar(15)
	log_level	varchar(15)
	message_id	varchar(15)
	message_details	varchar(555)
	log_date	char(10)

Instructor demonstration

Partitioning performance
impacts (10 mins)

Bucketing

Another technique for dividing data into manageable chunks

- While partitions logically subdivide data into folders, buckets control the number of files within a folder.
- With bucketing, you choose a fixed number of buckets/files up front.
- Decide bucket count based on desired/ideal file size
- A consistent algorithm is used to determine the bucket assignment.
 - Bucketing keeps data that has the same key value in the same bucket.
 - This can improve performance, as matching data is always stored together in the same bucket.

BY ITSELF

```
.../cat/sch/hist_tbl
|-- bucketfile0
|-- bucketfile1
|-- bucketfile2
```

w/PARTITIONING

```
.../cat/sch/hist_tbl
|-- year=2020
|   |-- bucketfile0
|   |-- bucketfile1
|   |-- bucketfile2
|-- year=2021
|   |-- bucketfile0
|   |-- bucketfile1
|   |-- bucketfile2
|-- year=2022
|   |-- bucketfile0
|   |-- bucketfile1
|   |-- bucketfile2
```

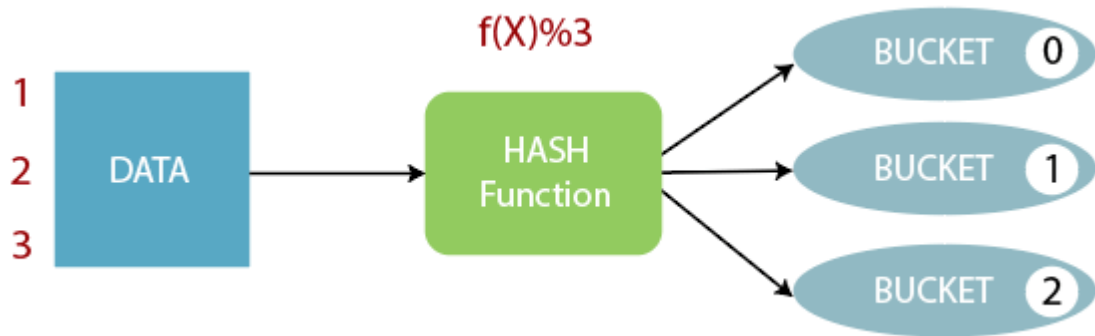
Define bucketing details at table creation

```
CREATE TABLE hive.web.page_views (  
    view_time timestamp,  
    user_id bigint,  
    page_url varchar,  
    ds date,  
    country varchar  
)  
  
WITH (  
    format = 'ORC',  
    partitioned_by = ARRAY['ds', 'country'],  
    bucketed_by = ARRAY['user_id'],  
    bucket_count = 50  
);
```

Hashing: The bucket assignment algorithm

A hash function is a way to map data of arbitrary size into a fixed number of buckets.

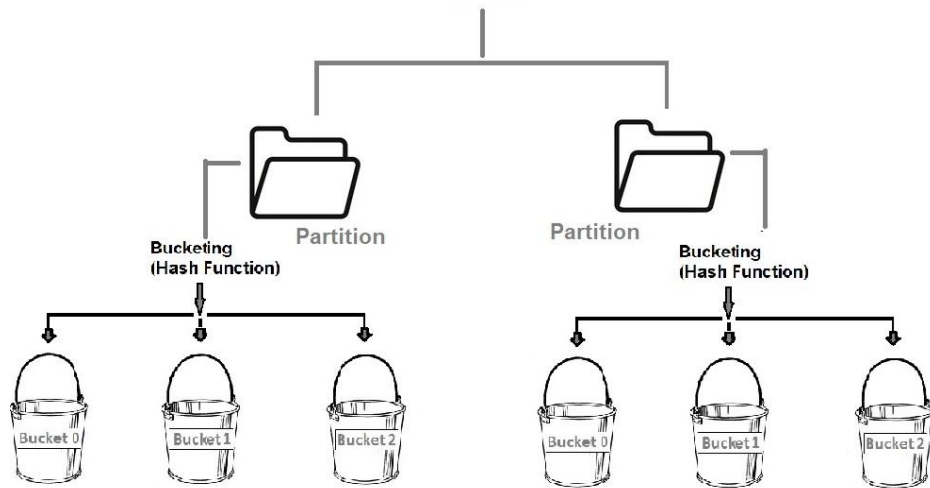
- An example of a simple hash function is “modulo,” which provides the remainder after dividing two values.
- **For example:** $4 \bmod 3 = 1$ because 4 divided by 3 leaves a remainder of 1.
- Hash functions are used to map values in joins to buckets in a hash table, or to map them to specific server nodes for processing.



Bucketing vs. partitioning

Each solves a different problem.

- Both are useful to limit the amount of data to read.
- **Partitions:**
 - Implemented using folder structures
 - Work best on low cardinality columns that have fairly uniform distribution
- **Buckets:**
 - Implemented by splitting files
 - A way to continue to subdivide data that may not make sense to partition, such as high cardinality and/or highly skewed columns
- BE CAREFUL OF CREATING SMALL FILES USING EITHER/BOTH OF THESE FEATURES



Additional strategy: Sorting

Total order sorting for a table or partition

- Sorting all of the files for a specific table, or partition, on heavily queried column(s) allows for index-like behavior.
- Sorting uses the statistics associated with each file, including min & max values, and checks to see if a particular WHERE selection is either:
 - Potentially in the file
 - Definitely not in the file
- Like partitioning & bucketing, this is another way to intelligently limit the amount of data it has to read

select * from tbl where id between 555 and 777;		
Unsorted		Sorted
Field: id		Field: id
min 101, max 999	file A	min 101, max 122
min 150, max 999	file B	min 123, max 244
min 101, max 955	file C	min 245, max 490
min 110, max 988	file D	min 490, max 829
min 101, max 999	file E	min 830, max 999
	READ	
	SKIPPED	

Hands-on exercise

Exploring table partitioning &
bucketing (40 mins)

Lesson summary

Data lake performance: Partitioning & bucketing

1. Partitioning separates data into folders based on a low cardinality column's value and allows a WHERE clause to determine a subset of these folders to be read.
2. Using a hashing algorithm, bucketing ensures all data with the same value of a high cardinality column is persisted together in the same file thus allowing a WHERE clause to determine which file(s) to read.
3. Total order sorting across files allows the statistics associated with columnar formats to be utilized by the query engine to quickly stop reading a file if a particular value, or range of values, could not be present.
4. A bloom filter is a probabilistic technique that can allow the majority of files to not have to be read when querying for rare values in high cardinality columns.